

Stack

Algorithm questions with their time complexity + solution

1. Reverse a String Using a Stack

Problem

Given a string, reverse it using a stack.

Algorithm

Create an empty stack

For each character:

Push onto stack

While stack not empty:

Pop and print

Time Complexity : $O(n)$ if n is the length of the string

2. Check Whether a String is a Palindrome Using a Stack

Problem

Determine whether a string reads the same forwards and backwards.

Algorithm

Push all characters into stack

For each character in original string:

if character $\neq$ Pop():

return Not Palindrome

return Palindrome

Time Complexity : $O(n)$ if n is the length of the string

3. Find the Sum of All Elements in a Stack

Problem

Given a stack containing N integers, find the sum of all elements.

Algorithm

```
sum = 0
```

```
While stack is not empty
```

```
sum = sum + Pop()
```

```
Print sum
```

Time Complexity : $O(n)$

4. Insert an Element at the Bottom of a Stack

Problem

Given a stack and an element X, insert X at the bottom of the stack.

Algorithm

```
Create an empty temporary stack T
```

```
While stack is not empty
```

```
T.push(S.pop())
```

```
S.push(X)
```

```
While T is not empty
```

```
S.push(T.pop())
```

Time Complexity : $O(n)$

5. Search for an Element in a Stack

Problem

Given a stack and an element X, determine whether X is present in the stack.

Algorithm

```
SEARCH_STACK(S, X)
```

```
while S is not empty do
```

```
element ← POP(S)
```

```
if element = X then
```

```
    print "Found"
```

```
    return
```

```
print "Not Found"
```

Time Complexity : $O(n)$

Programming questions + solution

1. Decimal to Binary Conversion Using a Stack

Question

Write a Python program to convert a decimal number to binary using a stack.

Example

Input: 13
Output: 1101

Program

```
num = int(input("Enter the decimal number :"))
stack = []

while num > 0:
    stack.append(num % 2)
    num = num // 2
while stack:
    print(stack.pop(), end="")
```

2. Reverse a List Using a Stack

Question

Reverse a list using a stack.

Example

Input:
[10, 20, 30, 40]
Output:
[40, 30, 20, 10]

Program

```
arr = [10, 20, 30, 40]
stack = []

for item in arr:
    stack.append(item)

reversed_list = []
while stack:
    reversed_list.append(stack.pop())

print(reversed_list)
```

3. Check Balanced Parentheses

Question

Check whether an expression contains balanced parentheses.

Example

Input:

$((a+b)*(c-d))$

Output:

Balanced

Program

```
expr = "((a+b)*(c-d))"

stack = []
balanced = True

for ch in expr:
    if ch == '(':
        stack.append(ch)
    elif ch == ')':
        if len(stack) == 0:
            balanced = False
            break
        stack.pop()

if balanced and len(stack) == 0:
    print("Balanced")
Else:
    print("Not Balanced")
```

Queue

Algorithm questions with their time complexity + solution

1. Count the Number of Elements in a Queue

Problem Given a queue containing N elements, find the number of elements present in the queue.

Algorithm

```
count = 0
while queue is not empty
  Dequeue()
  count++
print count
Time Complexity :  $O(n)$ 
```

2. Display All Elements of a Queue

Problem Write an algorithm to display all elements of a queue from front to rear.

Algorithm

```
while queue is not empty
  print(Front)
  Dequeue()
Time Complexity :  $O(n)$ 
```

3. Search for an Element in a Queue

Problem Determine whether a given element X is present in a queue.

Algorithm

```
while queue is not empty
  item = Dequeue()
  if item == X
    print "Found"
    stop
print "Not Found"
```

Time Complexity : $O(n)$

4. Find the Rear Element of a Queue

Problem

Write an algorithm to display the last (rear) element of a queue without removing it.

Algorithm

```
if queue is empty
  print "Queue Empty"
else
```

print Rear

Time Complexity : $O(1)$

5. ATM Queue System

Problem:

Customers arrive at an ATM and wait in a queue. The customer who arrives first is served first (FIFO).

Algorithm

ATM_QUEUE()

Create an empty queue.

Enqueue Ravi.

Enqueue Priya.

Enqueue Kumar.

Dequeue Ravi and serve him at the ATM.

Dequeue Priya and serve her at the ATM.

Dequeue Kumar and serve him at the ATM.

Stop.

Time Complexity : $O(n)$ for n customers

Programming questions + solution

Question 1

Write a Python program to simulate a printer queue using an array. Add three print jobs and process them in FIFO order.

Program

```
queue = []
front = 0
# Enqueue operations
queue.append("Doc1")
queue.append("Doc2")
queue.append("Doc3")
print("Print Jobs:", queue)
# Dequeue and process jobs
while front < len(queue):
    print("Printing:", queue[front])
```

```
front += 1

print("All jobs completed")
```

Question 2

Write a Python program to simulate a customer service queue (normal queue using list). Add 3 customers and serve the first customer.

Program

```
queue = []

# Enqueue

queue.append("Alice")
queue.append("Bob")
queue.append("Charlie")
print("Queue:", queue)

# Dequeue

served = queue.pop(0)
print("Served:", served)

print("Remaining Queue:", queue)
```

Question 3

A railway station can add coaches at both the front and rear of a train. Write a Python program to simulate this using a deque.

Program

```
train = []

# Add coach at rear

train.append("C1")
train.append("C2")

# Add coach at front

train.insert(0, "Engine")

print("Train:", train)

# Remove front coach

train.pop(0)

print("After removing front coach:")

print(train)
```

Output

Train: ['Engine', 'C1', 'C2']

After removing front coach:

['C1', 'C2']

Question 4

Write a Python program using a circular queue to rotate songs in a playlist. When a song finishes, move to the next song and continue from the beginning after the last song.

Program

```
playlist = ["Believer", "Shape of You", "Perfect", "Faded", "Senorita", "Counting Stars"]
current = 0
while True:
    print("Now Playing:", playlist[current])
    choice = input("Press Enter for next song or type STOP: ")
    if choice.upper() == "STOP":
        print("Playlist Stopped")
        break
    current = (current + 1) % len(playlist)
```

Question 5

Write a Python program to simulate a hospital emergency room using a priority queue. Patients with higher priority are treated first.

```
1 = Critical
2 = Serious
3 = Normal
```

Program

```
patients = [["John", 2], ["Mary", 1], ["David", 3], ["Sam", 2]]
while len(patients) > 0:
    highest = 0
    for i in range(1, len(patients)):
        if patients[i][1] < patients[highest][1]:
            highest = i
    patient = patients.pop(highest)
    print("Treating:", patient[0], "Priority:", patient[1])
```

Output

Treating: Mary Priority: 1

Treating: John Priority: 2

Treating: Sam Priority: 2

Treating: David Priority: 3

Lists

Algorithm questions with their time complexity + solution

1. Count Number of Nodes

Algorithm

```
count = 0
current = head
```

```
while current is not None:
    count += 1
    current = current.next
```

Time Complexity = $O(n)$

2. Search for an Element

Algorithm

```
current = head
key = 30
```

```
while current is not None:
    if current.data == key:
        print("Found")
        break
    current = current.next
..
```

Time Complexity = $O(n)$

3. Find Maximum Element

Algorithm

```
max_value = head.data
current = head.next
```

```
while current is not None:
    if current.data > max_value:
        max_value = current.data
    current = current.next
```

```
print(max_value)
```

Time Complexity = $O(n)$

4. Find the Last Node

Algorithm

```
current = head
```

```
while current.next is not None:
    current = current.next

print("Last Node =", current.data)
```

Time Complexity = $O(n)$

Programming questions + solution

1. Singly Linked List: Reverse the List

Question

Write a Python program to:

1. Create a singly linked list.
2. Insert multiple elements at the end.
3. Reverse the linked list.
4. Display the list before and after reversal.

Program

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next is not None:
            current = current.next

        current.next = new_node

    def reverse(self):
        previous = None
        current = self.head

        while current is not None:
            next_node = current.next
            current.next = previous
            previous = current
            current = next_node
```

```
previous = current
current = next_node
```

```
self.head = previous
```

```
def display(self):
    current = self.head
```

```
while current is not None:
    print(current.data, end=" -> ")
    current = current.next
```

```
print("None")
```

```
linked_list = SinglyLinkedList()
```

```
for value in [10, 20, 30, 40, 50]:
    linked_list.insert_at_end(value)
```

```
print("Original linked list:")
linked_list.display()
```

```
linked_list.reverse()
```

```
print("Reversed linked list:")
linked_list.display()
```

2. Singly Linked List: Remove Duplicate Elements

Question

Write a Python program to remove duplicate elements from an **unsorted singly linked list**.

```
class Node:
```

```
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class SinglyLinkedList:
```

```
    def __init__(self):
        self.head = None
```

```
    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = new_node
```

```
    def remove_duplicates(self):
```

```

if self.head is None:
    return

    seen = set()
    current = self.head
    previous = None

    while current is not None:
        if current.data in seen:
            previous.next = current.next
        else:
            seen.add(current.data)
            previous = current
        current = current.next

def display(self):
    current = self.head

    while current is not None:
        print(current.data, end=" -> ")
        current = current.next
    print("None")

```

```

linked_list = SinglyLinkedList()

for value in [10, 20, 10, 30, 20, 40, 30]:
    linked_list.insert_at_end(value)

print("Before removing duplicates:")
linked_list.display()

linked_list.remove_duplicates()

print("After removing duplicates:")
linked_list.display()

```

3. Doubly Linked List: Browser History Navigation

Question

Write a Python program using a doubly linked list to simulate browser history. The program should support:

- Visiting a new webpage
- Moving backward
- Moving forward
- Displaying the current webpage

Program

```

class WebPage:
    def __init__(self, url):
        self.url = url
        self.previous = None

```

```
self.next = None
```

```
class BrowserHistory:
```

```
def __init__(self):
```

```
    self.current = None
```

```
def visit(self, url):
```

```
    new_page = WebPage(url)
```

```
    if self.current is None:
```

```
        self.current = new_page
```

```
    else:
```

```
        # Remove the forward history
```

```
        self.current.next = None
```

```
        new_page.previous = self.current
```

```
        self.current.next = new_page
```

```
        self.current = new_page
```

```
    print("Visited:", self.current.url)
```

```
def go_back(self):
```

```
    if self.current is None:
```

```
        print("No page is currently open.")
```

```
    elif self.current.previous is None:
```

```
        print("No previous page available.")
```

```
    else:
```

```
        self.current = self.current.previous
```

```
        print("Moved back to:", self.current.url)
```

```
def go_forward(self):
```

```
    if self.current is None:
```

```
        print("No page is currently open.")
```

```
    elif self.current.next is None:
```

```
        print("No forward page available.")
```

```
    else:
```

```
        self.current = self.current.next
```

```
        print("Moved forward to:", self.current.url)
```

```
def show_current_page(self):
```

```
    if self.current is None:
```

```
        print("No page is currently open.")
```

```
    else:
```

```
        print("Current page:", self.current.url)
```

```
browser = BrowserHistory()
```

```
browser.visit("google.com")
```

```
browser.visit("wikipedia.org")
```

```
browser.visit("github.com")
```

```
browser.show_current_page()
```

```
browser.go_back()
```

```
browser.go_back()
```

```
browser.go_forward()
```

```
browser.visit("microsoft.com")
```

```
browser.go_forward()
```

```
browser.show_current_page()
```

4. Circular Linked List: CPU Round-Robin Scheduling

Question

Write a Python program using a circular linked list to simulate round-robin CPU scheduling. Each process should get one turn to execute. After the last process, execution should return to the first process.
Run the scheduler for a specified number of cycles.

Program

```
class ProcessNode:
    def __init__(self, process_name):
        self.process_name = process_name
        self.next = None

class RoundRobinScheduler:
    def __init__(self):
        self.head = None
        self.tail = None

    def add_process(self, process_name):
        new_process = ProcessNode(process_name)

        if self.head is None:
            self.head = new_process
            self.tail = new_process
            new_process.next = self.head
        else:
            self.tail.next = new_process
            self.tail = new_process
            self.tail.next = self.head

    def display_processes(self):
        if self.head is None:
            print("No processes available.")
            return

        current = self.head

        while True:
            print(current.process_name, end=" -> ")
            current = current.next
```

```

        if current == self.head:
            break
    print("Back to", self.head.process_name)

def run_scheduler(self, number_of_turns):
    if self.head is None:
        print("No processes available.")
    return

current = self.head

for turn in range(1, number_of_turns + 1):
    print(
        "Turn",
        turn,
        ": Executing",
        current.process_name
    )
    current = current.next

scheduler = RoundRobinScheduler()

scheduler.add_process("Process 1")
scheduler.add_process("Process 2")
scheduler.add_process("Process 3")
scheduler.add_process("Process 4")

print("Processes in the circular list:")
scheduler.display_processes()

print("\nCPU execution:")
scheduler.run_scheduler(10)

```